

4. PageRank

2026-04-13, 21:18

Tags: [ML](#) [GNN](#) [Neural Network](#)

Represent the graph as page

PageRank

- Web as a graph
See Node = web pages
See Edges = hyperlinks

The key here is the Link Analysis Approaches

- PageRank
- Personalized PageRank
- Random Walk with Restarts

The idea, link as vote

Page is more important if it has more links

■ Idea: Links as votes

- Page is more important if it has more links
 - In-coming links? Out-going links?

■ Think of in-links as votes:

- www.stanford.edu has 23,400 in-links
- thispersondoesnotexist.com has 1 in-link

■ Are all in-links equal?

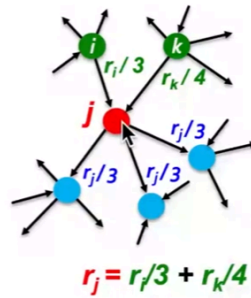
- Links from important pages count more
- Recursive question!

**A "vote" from an important page is worth more:

- each link's vote is proportional to the importance of its source page
- if page i with importance r_i has d_i out-links, each link get $\frac{r_i}{d_i}$ votes
- node j 's importance r_j is the sum of the votes on its in-links

- A “vote” from an important page is worth more:

- Each link’s vote is proportional to the **importance** of its source page
- If page i with importance r_i has d_i out-links, each link gets r_i/d_i votes
- Page j ’s own importance r_j is the sum of the votes on its in-links



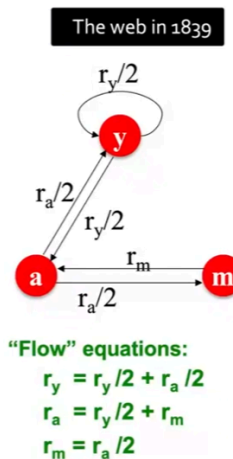
1. Using the "FLOW" model

- A page is important if it is pointed to by other important pages
- Define “rank” r_j for node j

$$r_j = \sum_{i \rightarrow j} \frac{r_i}{d_i}$$

d_i ... out-degree of node i

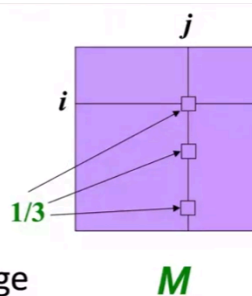
You might wonder: Let’s just use Gaussian elimination



2. Matrix formulation

- **Stochastic adjacency matrix M**

- Let page j have d_j out-links
- If $j \rightarrow i$, then $M_{ij} = \frac{1}{d_j}$
 - M is a **column stochastic matrix**
 - Columns sum to 1



- **Rank vector r** : An entry per page

- r_i is the importance score of page i
- $\sum_i r_i = 1$

- **The flow equations can be written**

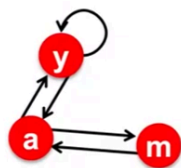
$$r = M \cdot r$$

$$r_j = \sum_{i \rightarrow j} \frac{r_i}{d_i}$$

So basically, M_{ij} represent the node j is connected with i . and the importance= $1/d_j$. Furthermore, each column of M sum up to be one, because M is a stochastic matrix.

Rank vector r is importance score of page i . It sum up to be 1, but it can dynamically adjust the value.

For example:



	r_y	r_a	r_m
r_y	$\frac{1}{2}$	$\frac{1}{2}$	0
r_a	$\frac{1}{2}$	0	1
r_m	0	$\frac{1}{2}$	0

$$\begin{aligned} r_y &= r_y/2 + r_a/2 \\ r_a &= r_y/2 + r_m \\ r_m &= r_a/2 \end{aligned}$$

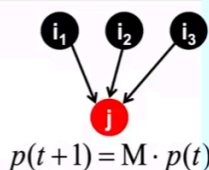
$$\begin{matrix} \begin{matrix} r_y \\ r_a \\ r_m \end{matrix} \\ \mathbf{r} \end{matrix} = \begin{matrix} \begin{matrix} \frac{1}{2} & \frac{1}{2} & 0 \\ \frac{1}{2} & 0 & 1 \\ 0 & \frac{1}{2} & 0 \end{matrix} \\ \mathbf{M} \end{matrix} \begin{matrix} \begin{matrix} r_y \\ r_a \\ r_m \end{matrix} \\ \mathbf{r} \end{matrix}$$

In this case, it reveals the stationary distribution through the random walk. And here is the illustration:

■ **Where is the surfer at time $t+1$?**

- Follow a link uniformly at random

$$\mathbf{p}(t+1) = \mathbf{M} \cdot \mathbf{p}(t)$$



- Suppose the random walk reaches a state

$$\mathbf{p}(t+1) = \mathbf{M} \cdot \mathbf{p}(t) = \mathbf{p}(t)$$

then $\mathbf{p}(t)$ is **stationary distribution** of a random walk

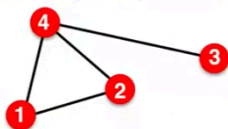
- Our original rank vector $\mathbf{r}$ satisfies $\mathbf{r} = \mathbf{M} \cdot \mathbf{r}$

- So, $\mathbf{r}$ is a **stationary distribution for the random walk**

and it is connected with eigenvector centrality:

- Recall from lecture 2 (eigenvector centrality), let

$\mathbf{A} \in \{0, 1\}^{n \times n}$ be an adj. matrix of undir. graph:



$$\mathbf{A} = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}$$

- Eigenvector of adjacency matrix:

vectors satisfying $\lambda \mathbf{c} = \mathbf{A} \mathbf{c}$

- $\mathbf{c}$: eigenvector; λ : eigenvalue

Eigenvector Formulation

- The Flow equation:

$$\mathbf{1} \cdot \mathbf{r} = \mathbf{M} \cdot \mathbf{r}$$

which looks like $(\mathbf{A}x = \lambda x)$, where $\lambda = 1$ and $x = \mathbf{r}$.

in this case, $\mathbf{r}$ is the eigenvector of $\mathbf{M}$ with eigenvalue 1.

Geometrically, the vector is not even being stretch (since is it multiplied by 1)

Power iteration

- Solve for r by assign each node an initial page rank
- repeat until convergence
 - Calculate the page rank of each node

■ **Given a web graph with N nodes, where the nodes are pages and edges are hyperlinks**

■ **Power iteration:** a simple iterative scheme

■ Initialize: $\mathbf{r}^0 = [1/N, \dots, 1/N]^T$

■ Iterate: $\mathbf{r}^{(t+1)} = \mathbf{M} \cdot \mathbf{r}^t$

■ Stop when $|\mathbf{r}^{(t+1)} - \mathbf{r}^t|_1 < \epsilon$

$$r_j^{(t+1)} = \sum_{i \rightarrow j} \frac{r_i^{(t)}}{d_i}$$

$d_i \dots$ out-degree of node i

$|\mathbf{x}|_1 = \sum_1^N |x_i|$ is the **L1** norm

Can use any other vector norm, e.g., Euclidean

About 50 iterations is sufficient to estimate the limiting solution.

An example:

■ **Power Iteration:**

■ Set $r_j \leftarrow 1/N$

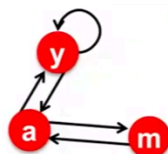
■ 1: $r'_j \leftarrow \sum_{i \rightarrow j} \frac{r_i}{d_i}$

■ 2: If $|r - r'| > \epsilon$:

■ $r \leftarrow r'$

■ 3: go to 1

■ **Example:**



	y	a	m
y	1/2	1/2	0
a	1/2	0	1
m	0	1/2	0

$$\begin{aligned} r_y &= r_y/2 + r_a/2 \\ r_a &= r_y/2 + r_m \\ r_m &= r_a/2 \end{aligned}$$

$$\begin{pmatrix} r_y \\ r_a \\ r_m \end{pmatrix} = \begin{pmatrix} 1/3 \\ 1/3 \\ 1/3 \end{pmatrix} \begin{pmatrix} 1/3 \\ 3/6 \\ 1/6 \end{pmatrix} \begin{pmatrix} 5/12 \\ 1/3 \\ 3/12 \end{pmatrix} \begin{pmatrix} 9/24 \\ 11/24 \\ 1/6 \end{pmatrix} \dots \begin{pmatrix} 6/15 \\ 6/15 \\ 3/15 \end{pmatrix}$$

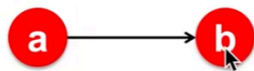
Iteration 0, 1, 2, ...

Two problems

1. Some pages are dead ends (have no outlinks)
2. Spider traps (all out-links are within the group)

• Dead End

■ **The "Dead end" problem:**



$$r_j^{(t+1)} = \sum_{i \rightarrow j} \frac{r_i^{(t)}}{d_i}$$

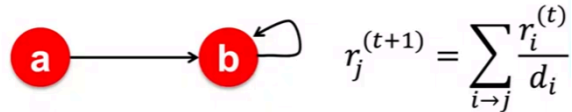
■ **Example:**

	Iteration: 0,	1,	2,	3...
r_a	1	0	0	0
r_b	0	1	0	0

The random walker just dead at the end, it just produce 0 and leak out the graph.

- Spider trap

- The “Spider trap” problem:



- Example:

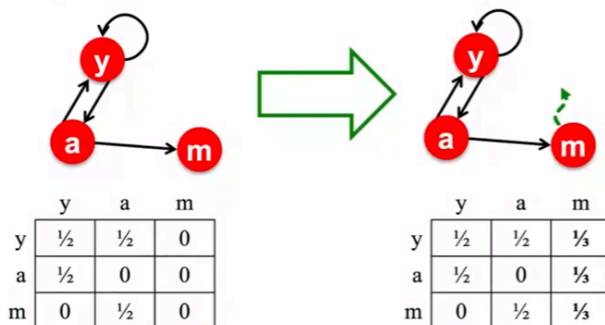
Iteration:	0,	1,	2,	3...
r_a	1	0	0	0
r_b	0	1	1	1

The random walker is trap at one node and cannot get out

Solution to these problem

- For deadend traps: *follow random teleport links with total probability 1.0 from dead-ends*

- Adjust matrix accordingly

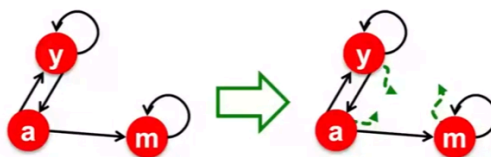


(The idea behind this is to make the matrix column stochastic by always teleporting when there is nowhere else to go)

- For spider traps: *at each time step, the random surfer has two options*

1. with prob. β follow a link at random
2. with prob. $1 - \beta$ jump to a random page
3. common values for β are in the range of 0.8 to 0.9

- Surfer will teleport out of spider trap within a few time steps



The solved result:

- **PageRank equation** [Brin-Page, '98]

$$r_j = \sum_{i \rightarrow j} \beta \frac{r_i}{d_i} + (1 - \beta) \frac{1}{N}$$

- **The Google Matrix G :**

$[1/N]_{N \times N} \dots N$ by N matrix
where all entries are $1/N$

$$P = \beta M + (1 - \beta) \left[\frac{1}{N} \right]_{N \times N}$$

- **We have a recursive problem: $r = G \cdot r$**

And the Power method still works!

- **What is β ?**

- In practice $\beta = 0.8, 0.9$ (make 5 steps on avg., jump)

Topic specific PageRank --> Personalized PageRank

In this case, it only

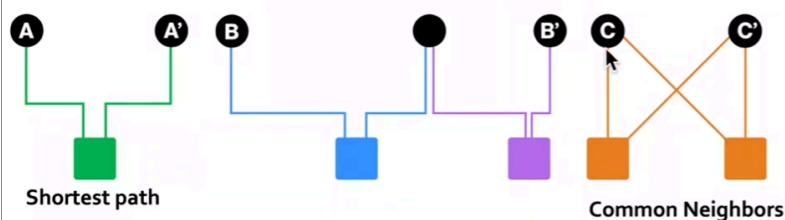
1. Teleport to a specific set of nodes
2. nodes can have different probabilities of the surfer landing there

$$S = [0.1, 0, 0.2, 0.5, 0, 0, 0.1, 0.1]$$

Random Walk with restarts

Purpose: Node proximity measurement

- **Which is more related A, A' , B, B' or C, C' ?**



In this case, it is obvious that A and A' are less connected compare to C and C' , since C and C' both are cross connected, sharing common neighbors

Proximity on Graphs

■ PageRank:

- Ranks nodes by “importance”
- Teleports with uniform probability to any node in the network

■ Personalized PageRank:

- Ranks proximity of nodes to the teleport nodes S

■ Proximity on graphs:

- **Q:** What is most related item to **Item Q**?
- **Random Walks with Restarts**
 - Teleport back to the starting node: $S = \{Q\}$

It is asking, what if instead of teleporting everywhere equally, you only teleport to a specific set of pages S ? In the restarting scenario, everytime after teleportation, the walker wake up and back to Q . In this scenario, the Node Q and nodes related to Q will have higher importance, as it has higher number of incoming nodes

Core idea:

Given: A set of **QUERY_NODES** (the node(s) you care about)

Do this repeatedly:

1. Start at a QUERY_NODE
2. Walk to a random neighbor → record +1 visit to that node
3. At each step:
 - With probability $(1 - \text{ALPHA})$: keep walking randomly
 - With probability ALPHA : restart back to a QUERY_NODE
4. Repeat for many many steps

Result: Count up visits to each node. Higher visit count = more related to your query.

What ALPHA Controls

α = restart probability

High α	Low α
Restart often	Rarely restart
Stay close to query node	Explore far away
Measures tight proximity	Measures loose proximity

It's a knob between "local" and "global" similarity.

Why Visit Count = Proximity

Think about it intuitively:

- The surfer **always restarts from Q**
- So nodes directly connected to Q get visited constantly
- Nodes far away are rarely reached before a restart happens
- Therefore **visit frequency naturally captures graph distance** from Q

Concrete Example

Say $Q = A$ in this graph:

```

A - B - C
|       |
D - E - F

```

Starting from A, the surfer explores outward. With restarts always going back to A:

- **B, D** get visited very often (1 hop from A)
 - **C, E** get visited sometimes (2 hops)
 - **F** gets visited rarely (3 hops)
- So the ranking by proximity to A would be: $B, D > C, E > F$

Matrix Factorization and Node Embedding

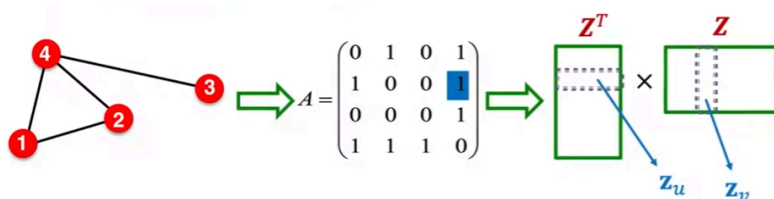
The simplest node similarity:

Node u, v are similar if they are connected!

Which means: $Z_v^T Z_u = A_{u,v}$.

Where u and v are two entries of the graph in the adjacency matrix A

Therefore, $Z^T Z = A$



However, We can't perfectly compress a graph into small vectors, so we find the best approximate compression by minimizing the L2 norm here—and this turns out to be identical to learning node embeddings via dot product similarity.

Matrix Factorization

- The embedding dimension d (number of rows in $\mathbf{Z}$) is much smaller than number of nodes n .
- Exact factorization $\mathbf{A} = \mathbf{Z}^T \mathbf{Z}$ is generally not possible
- However, we can learn $\mathbf{Z}$ approximately
- **Objective:** $\min_{\mathbf{Z}} \|\mathbf{A} - \mathbf{Z}^T \mathbf{Z}\|_2$
 - We optimize $\mathbf{Z}$ such that it minimizes the L2 norm (Frobenius norm) of $\mathbf{A} - \mathbf{Z}^T \mathbf{Z}$
 - Note in lecture 3 we used softmax instead of L2. But the goal to approximate $\mathbf{A}$ with $\mathbf{Z}^T \mathbf{Z}$ is the same.
- Conclusion: **inner product decoder with node similarity defined by edge connectivity is equivalent to matrix factorization of $\mathbf{A}$**

For Random Walk-based similarity

Volume of graph

$$vol(G) = \sum_i \sum_j A_{i,j}$$

Diagonal matrix D
 $D_{u,u} = \deg(u)$

context window size
 See Lec 3 slide 30:
 $T = |N_R(u)|$

Power of normalized adjacency matrix

Number of negative samples

$$\log \left(vol(G) \cdot \left(\frac{1}{T} \sum_{r=1}^T (D^{-1} A)^r \right) D^{-1} \right) - \log b$$

- **Node2vec** can also be formulated as a matrix factorization (albeit a more complex matrix)
- Refer to the paper for more detailed proofs.

Network Embedding as Matrix Factorization: Unifying DeepWalk, LINE, PTE, and node2vec, WSDM 18

Annotated Formula

$$\underbrace{\log}_{\text{log o PM}} \underbrace{vol(G)}_{\text{graph scale}} \cdot \underbrace{\frac{1}{T} \sum_{r=1}^T (D^{-1} A)^r}_{\text{avg. multi-hop walk pros}} \cdot \underbrace{D^{-1}}_{\text{degree normalie}} - \underbrace{\log b}_{\text{negative sample correction}}$$

Comparison: Simple Embeddings vs Node2Vec

Method	Matrix Factorized	Captures
Simple node embedding	$\mathbf{A}$	Direct connections only (1 hop)
DeepWalk	$\log \left(vol(G) \cdot \frac{1}{T} \sum_r (D^{-1} A)^r \cdot D^{-1} \right) - \log b$	Multi-hop neighborhood
Node2Vec	Same as DeepWalk but with biased walks (parameters p, γ)	Flexible local/global structure

Node2Vec is not doing anything fundamentally different from matrix factorization.

It is factorizing a **richer matrix** that encodes averaged multi-step random walk probabilities —

which is why it captures both local and global graph structure better than using A directly.

$$Z^T Z \approx \log \left(\text{vol}(G) \cdot \frac{1}{T} \sum_{r=1}^T (D^{-1} A)^r \cdot D^{-1} \right) - \log b$$